

Practical - 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

Theory:

1. How is JSON fetched, parsed, and rendered?

Answer: JSON data is fetched using the `fetch()` method. The response is converted into JavaScript data using `response.json()`. Then the data is displayed on the webpage using JavaScript and HTML.

2. Which array methods are used for search, filter, sort, and pagination?

Answer: `filter()` is used to search and filter the data.

`sort()` is used to arrange the data.

`slice()` is used for pagination to display a limited number of records at a time.

`map()` can be used to create and display multiple records.

3. How are loading and error states handled?

Answer: A loading message such as "Loading..." is shown while the JSON data is being fetched. If the data cannot be loaded, the `.catch()` method displays an error message.

4. How is modularity maintained in JavaScript files?

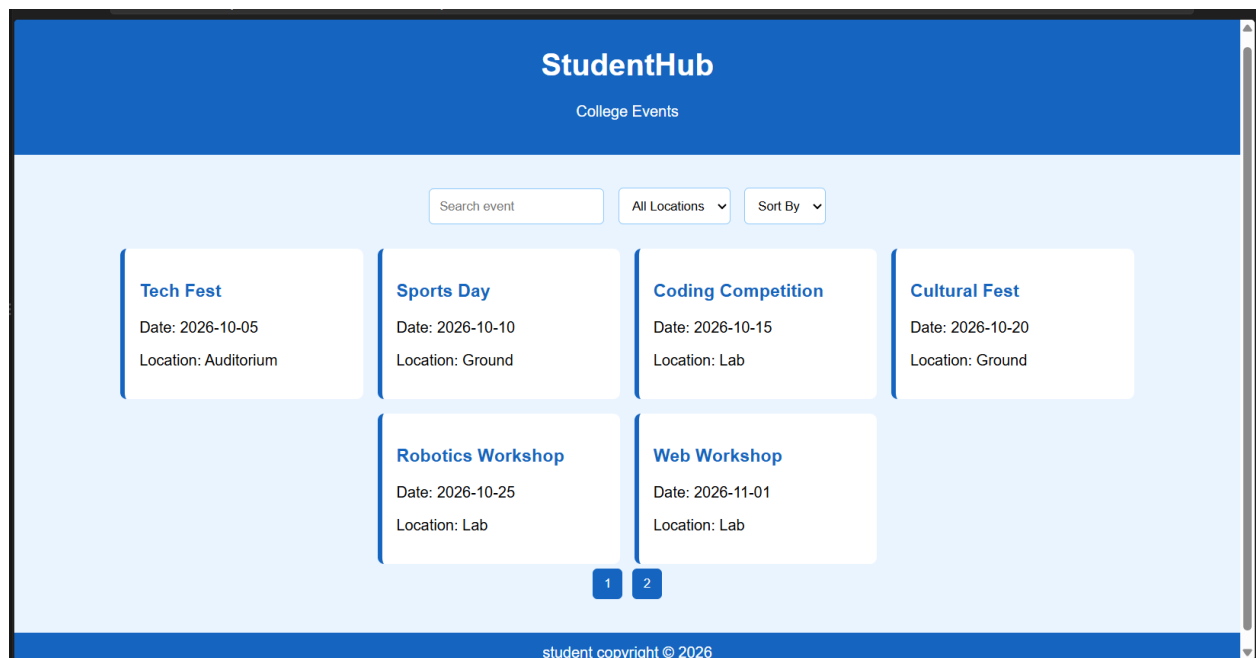
Answer: Modularity is maintained by keeping different tasks in separate functions, such as fetching data, displaying data, searching, filtering, sorting, and pagination. This makes the code easier to understand and modify.

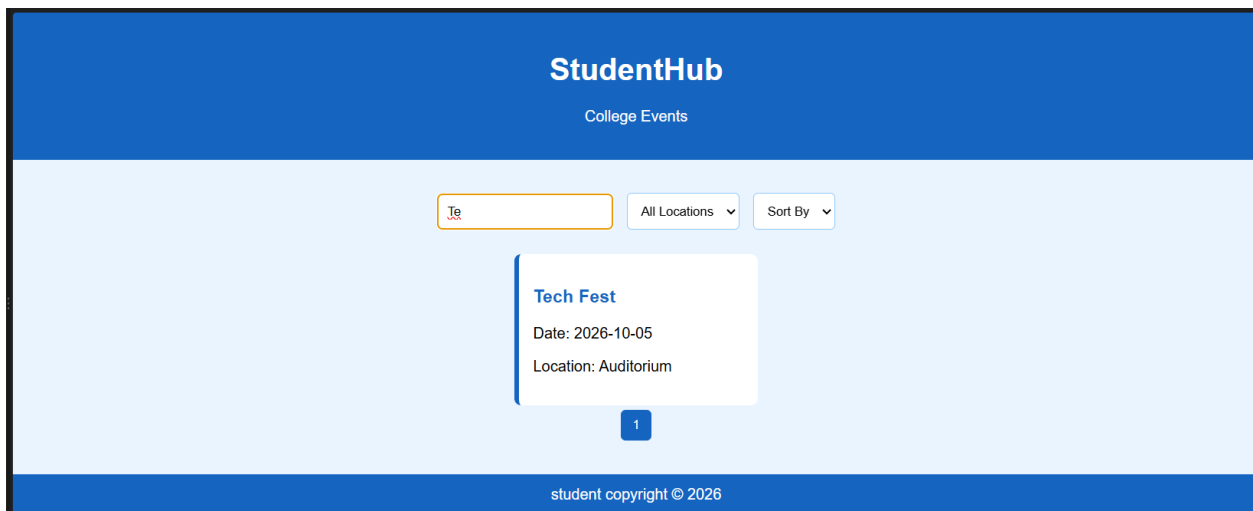
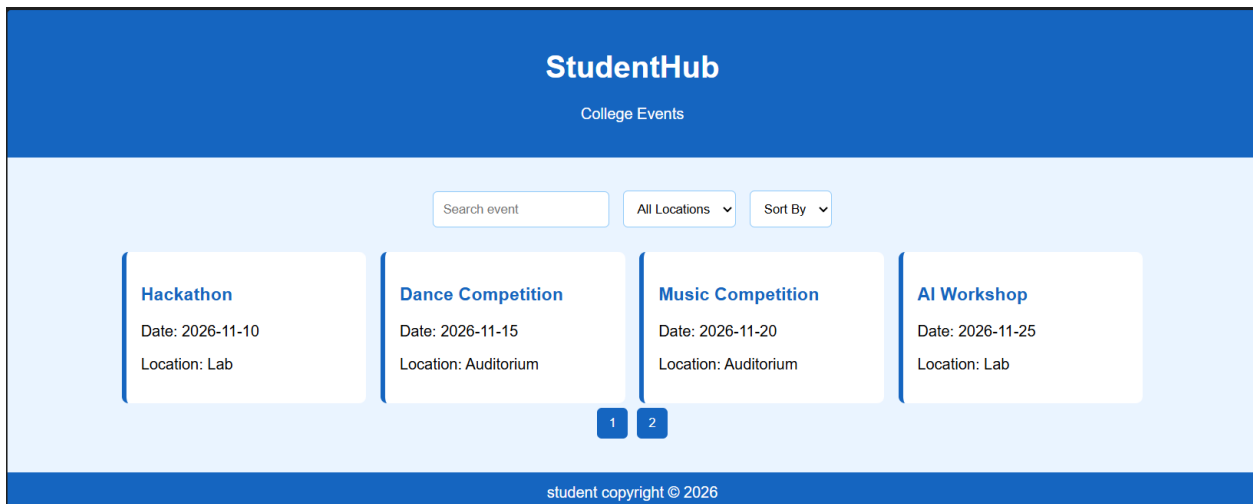
A. Implementation/ Output/Result (Keep only one option which is suitable to practical)

- `<html>` – Defines the complete HTML document.
- `<head>` – Contains information about the webpage, such as title and CSS.
- `<title>` – Sets the title shown on the browser tab.
- `<style>` – Used to add CSS styling to the webpage.
- `<body>` – Contains all the visible content of the webpage.
- `<header>` – Creates the top section containing the StudentHub name and heading.

- `<h1>` – Displays the main heading, "StudentHub".
- `<p>` – Displays paragraphs or small text, such as "College Events".
- `<main>` – Contains the main content of the webpage.
- `<input>` – Creates the search box for searching events.
- `<select>` – Creates dropdown menus for filtering and sorting.
- `<option>` – Provides choices inside the dropdown menus.
- `<div>` – Used to create sections/containers for displaying event cards and pagination.
- `<h3>` – Displays the event name as a heading.
- `<footer>` – Creates the bottom section containing the copyright text.
- `<script>` – Contains the JavaScript code used for Fetch API, searching, filtering, sorting, and pagination.
- `fetch()` → Gets JSON data.
- `json()` → Converts JSON response into JavaScript data.
- `filter()` → Searches and filters events.
- `sort()` → Sorts events.
- `slice()` → Used for pagination.
- `forEach()` → Displays each event.
- `addEventListener()` → Handles user actions like typing and selecting options.

A. Evidence of Work (Screenshots)





A. GitHub Repository Link:

<https://github.com/diyatolia1212/StudentHub/tree/ce7c39fbc46be406b3c0b7884cb2d428a5049c1d/PRACTICAL%206>

B. Challenges Faced:

- Understanding how to fetch JSON data using the Fetch API.
- Handling errors when the JSON file could not be loaded.
- Implementing search, filter, and sorting of event data.
- Creating pagination to display limited events on each page.
- Connecting the JSON data with the HTML page and displaying it properly.

Learning Outcomes:

- Learned how to use Fetch API to get external JSON data.
- Learned how to parse and display JSON data using JavaScript.
- Learned how to use filter(), sort(), slice() and forEach() methods.
- Learned how to create a dynamic webpage using JavaScript.
- Learned how to handle loading and error conditions.
- Improved understanding of HTML, CSS, JavaScript and JSON working together.

Faculty Remarks: (Keep this blank for faculty)